

From Pen to Pixel: Building a Custom OCR Pipeline to Digitize My Handwritten Journal Using PyTorch and AWS

Repository: https://github.com/orestasdulinskas/handwriting_recognition

Orestas Dulinskas
orestasdulinskas@gmail.com

Overview

Driven by a desire to digitize a year of handwritten journal entries and deepen my AI expertise, I embarked on building an end-to-end handwriting transcription tool. Over a period of approximately nine months, this project involved developing a custom CNN-LSTM Optical Character Recognition (OCR) model in PyTorch and an auto-annotation system using Faster R-CNN. The entire pipeline was then deployed on AWS Lambda as a serverless solution, accessible via an iOS Shortcut application, transforming my personal reflections into digital text. This innovative approach not only solved a personal challenge but also provided invaluable hands-on experience in advanced deep learning, computer vision, and cloud deployment.

The "Why": Motivation & Project Goals

I have been writing my journal for over a year. I started this to express myself creatively and support my mental health and self-discovery. For a more personal touch, I preferred handwriting my journal entries in an A5-size notebook. There I express my opinions, feelings and experiences. Having made many such entries in my journal over a year, I wanted to find a way to share some of them with people interested. I knew that the best way to do it would be to put them all into a blog.

Rewriting them in a digital form would be too tedious and time-consuming so I needed to make use of technology. Instead of using tools available already, having a deep interest in AI and having been enrolled in a Masters degree in data science, I decided to take on the challenge of building a handwriting transcription tool myself.

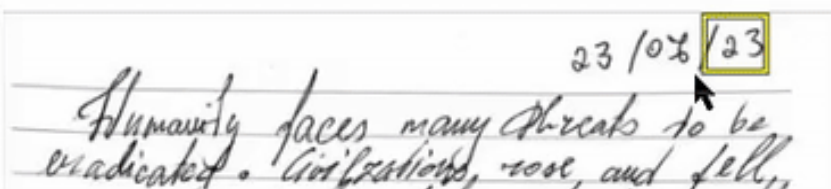
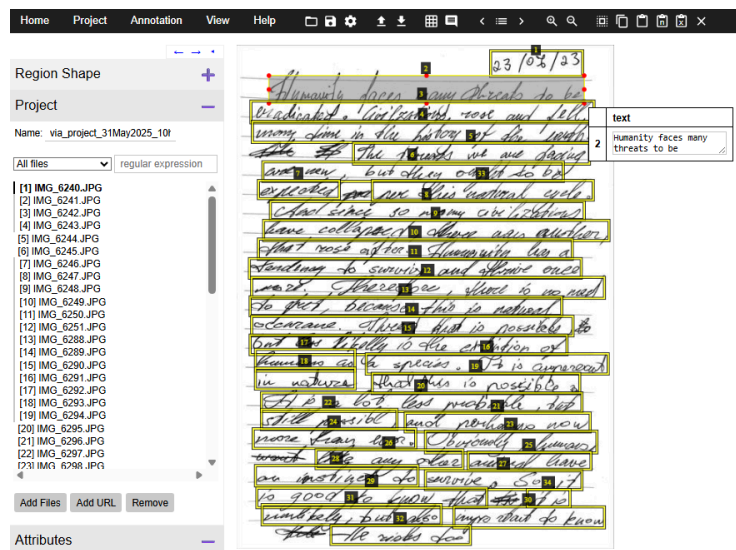
The idea was to take all the journal entries I have written so far and use them to train an AI model that would convert the handwritten text from images into digital text to be uploaded to my blog and at the same time, deepen my knowledge in Deep Learning applications.

Data: The Foundation of the Model

The data used in training AI models was purely created by me. It consisted of 60 scans of A5 notebook pages where I journaled my opinions, feelings and experiences. Each entry consisted of the title, date and the text of the journal entry. Each entry is pretty short, averaging about 178 words per entry. Scans were converted to grayscale images for clarity.

To prepare the data for training, all text within the images had to be annotated with a bounding box and text that tells what is written within the bounding box of that image. That has been done manually using the online [VGG Image Annotator \(VIA\) tool](#). It was chosen for ease of use and accessibility over the internet. I aimed to draw bounding boxes for each horizontal line as wide as possible so that they include as much text as possible. This often included 4.8 words on average, with as much as 11 words maximum. Words that have been brushed off have been left out of the bounding boxes. This made the annotation model ignore them as well.

Overall, the annotation process took about 3 months, aiming to annotate 1 entry per day during this period.



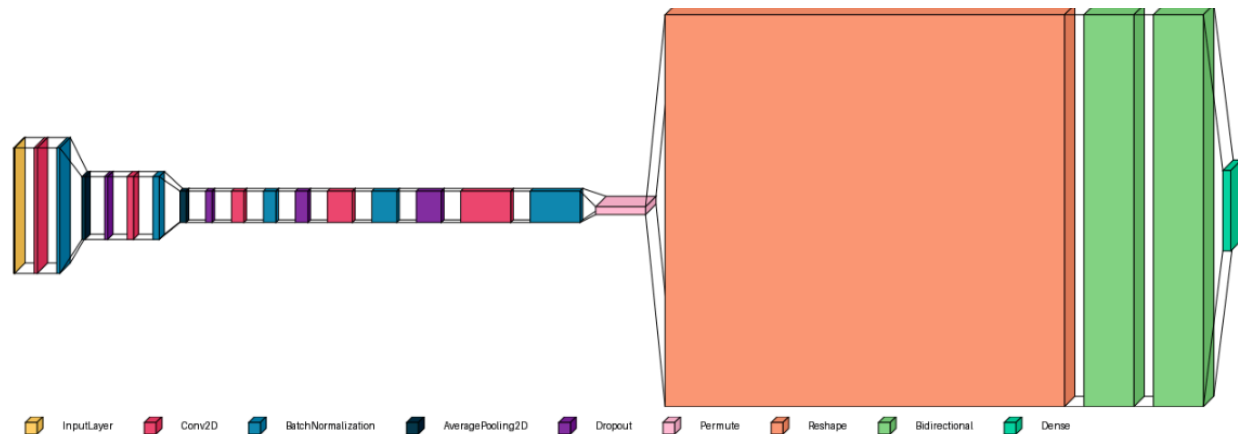
Model Development: The Core AI Work

OCR Model from Scratch (CNN-LSTM)

Model Architecture

The Optical Character Recognition (OCR) model makes use of a well-established and effective architecture with 5 Convolutional Neural Network (CNN) layers followed by 2 Bidirectional Long Short-Term Memory (BiLSTM) network layers. This treats handwriting recognition as a

sequence-to-sequence problem where the input is an image (a sequence of pixels) and the output is a sequence of characters.



Convolutional Neural Network automatically learns hierarchical spatial features. Each one of the 5 CNN layers progressively increases the number of feature maps, allowing the network to learn a wider variety of increasingly complex features at each stage. Early layers might detect edges or simple curves, while later layers detect more abstract character components. CNN layers also make use of Batch Normalisation, Average Pooling and Dropout:

- Batch Normalisation to improve training speed, stability and regularisation.
- Average Pooling makes the learned features more robust to small shifts or distortions.
- Dropout prevents overfitting by learning more robust and independent features. With 0.5 probability, half of the features are dropped.

After the CNN extracts a rich set of visual features from the handwriting image, these features are then flattened into a sequence. The LSTM takes this sequence of visual features and models the temporal dependencies between them. LSTMs are a type of RNN specifically designed to handle long-range dependencies, overcoming the vanishing/exploding gradient problems of vanilla RNNs.

- Bidirectional LSTM processes the input sequence in both forward and backward directions. This allows the model to leverage context from both the past and the future when predicting the current character.
- 2 layers make the model learn higher-level temporal representations. The first layer might learn basic character transitions, while the second might learn word-level context.
- A larger hidden size of 256 allows the LSTM to capture more complex patterns and store more information.
- The dropout layer specifically applied to the outputs of each LSTM layer prevents overfitting in the sequential part of the model. 0.7 probability applies strong regularization in the LSTM.

Finally, the final linear layer takes the hidden states from the LSTM and projects them into a probability distribution over the possible characters.

Extensive series of experiments were performed to pick best model parameters. 5 CNN layers were picked based on preliminary experiments where models with 3 layers underfit, while models with 7 layers showed signs of overfitting given our limited dataset and did not yield significant performance gains over 5 layers. Dropout of 0.5 in CNN layers and 0.7 in LSTM layers were selected after testing values from 0.3 to 0.7, as these provided the best regularization effect on the validation set CER.

Data for OCR model

To effectively train this CNN-LSTM architecture, meticulous data preparation was essential. To prepare data for training the OCR model, images were:

- Converted to grayscale which reduces computational complexity and is sufficient enough for handwriting data.
- The image height has been standardized to 45 pixels for a simpler and more consistent input shape.
- Images were normalised and channel dimensions were added to prepare for training using PyTorch.

Image augmentation was applied to increase the diversity of training data and make the model more robust to scan quality and minor distortions. It helps prevent overfitting and improves the model's generalization capabilities. The notebook applies augmentation 5 times for each original image.

The following augmentations were applied using the albumentations library:

- ShiftScaleRotate - Introduces slight variations in position, size, and rotation
- Perspective - Creates slight perspective distortions
- ElasticTransform - Introduces realistic handwriting variations by applying a smooth, elastic deformation
- GaussianBlur - Simulates slight blur
- RandomBrightnessContrast - Varies the brightness and contrast of the images
- RandomGamma - Adjusts the gamma correction
- GaussNoise - Add Gaussian noise
- ImageCompression - Simulates compression artefacts

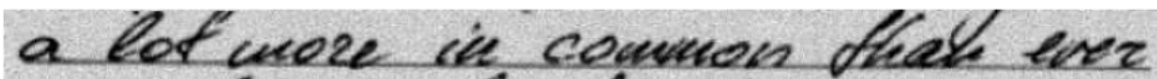
limbs and body



to his team and also work on a project



a lot more in common than ever



After augmentations are applied, the dataset size increases 5-fold resulting in 13056 bounding box images.

Label Encoding was done on the text to map a unique integer to each character found in the dataset. A special <PAD> token is added and mapped to 0, which is crucial for padding sequences. Then all sequences were padded to the maximum length using a special <PAD> token. This is necessary because LSTMs expect fixed-length input sequences within a batch.

Finally, the processed dataset, including the augmented ones, was split into training and validation sets, with 70% set for training and 30% for validation.

Loss Function & Optimizer

A custom loss function named FocalCTCLoss was defined for this model. It combines two key components:

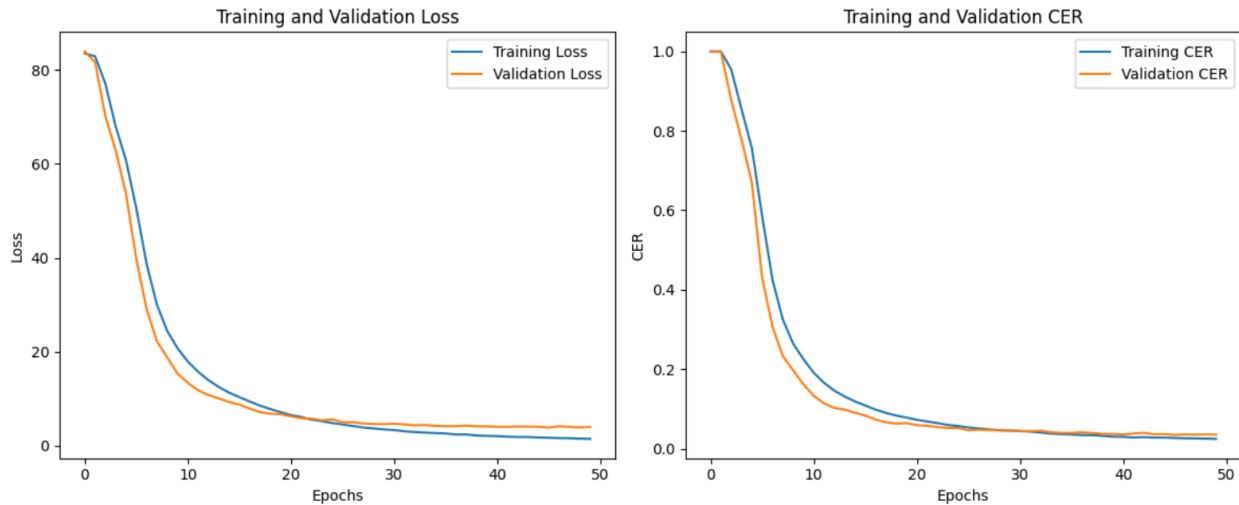
- Connectionist Temporal Classification (CTC) loss is the standard choice for sequence recognition tasks like OCR where there's no fixed alignment between the input (image features) and the output (character sequence). CTC allows the model to predict repetitions and blank tokens, which are then collapsed into the final transcription. It's ideal for handwriting where character spacing and length can vary greatly.
- Focal Loss addresses class imbalance by down-weighting well-classified examples, thus focusing training on hard, misclassified examples. While CTC itself handles sequence alignment, applying Focal Loss on top of individual CTC losses can be beneficial if certain characters or patterns are harder to recognize and the model tends to easily classify common ones. This helps the model pay more attention to challenging cases, potentially leading to better overall accuracy.

AdamW is used as an optimiser. It decouples weight decay (L2 regularization) from the gradient updates. This often leads to better generalization performance compared to traditional Adam with L2 regularization.

Additionally, a OneCycleLR learning rate scheduler was employed to speed up training and improve performance. It rapidly increases and then decreases the learning rate over the course of training. This cyclical approach helps the model escape saddle points, converge faster, and often leads to better final performance and generalization.

Metrics

To evaluate model performance, Focal CTC Loss and Character Error Rate (CER) are used as the primary metrics. CER measures the Levenshtein distance (number of insertions, deletions, and substitutions) between the predicted character sequence and the ground truth sequence, normalized by the length of the ground truth. A lower Loss and CER indicates better performance.



Trained model performance is inspected by comparing predictions and ground truth as below:

Sample 1:
Ground Truth: and respectful.
Prediction: and respectful.



Sample 2:
Ground Truth: one
Prediction: one



Sample 3:
Ground Truth: friends broke
Prediction: friends broke



Additional Training Details

The model was built and trained using Kaggle notebooks. They provide GPU access that speeds up the Neural Net training process. For this model 'GPU P100' was used for its high memory bandwidth and simplicity of using a single GPU. This allowed for a quick model training which took about 1 hour and 25 minutes.

Model training made use of Mixed Precision Training which allows the model to be trained using lower precision floating-point numbers (e.g., FP16) where possible. It significantly speeds up training and reduces memory usage, especially on modern GPUs.

Gradient Clipping was also employed to prevent exploding gradients, a common issue in RNNs (like LSTMs).

Lastly, an early stopping mechanism is implemented with a patience of 4 epochs, to prevent overfitting and save computational resources.

Challenges & Solutions

The CNN-LSTM model showed signs of overfitting, with training CER dropping to 1% but validation CER remaining at 5% after 10 epochs. High validation CER reduced transcription accuracy. To address this, I implemented extensive data augmentation (e.g., ElasticTransform, GaussianBlur) using the `augmentations` library, increasing the effective dataset size by 5x. Added dropout layers (0.5 for CNN, 0.7 for LSTM) and used early stopping with a patience of 4 epochs. Validation CER improved to 2.3%, and the model generalized better to unseen journal pages.

Auto-Annotation with Faster R-CNN

To make use of the trained OCR model in a streamlined manner, I also developed an auto-annotation model. It automatically annotated text from input images in a similar manner that I did when I manually annotated journal entries for training data preparation. This automates the detection of text regions in your handwritten pages. It serves as a bridge between the laborious manual annotation process and the high-accuracy requirements of the OCR pipeline.

Model Architecture

Given the limited set of manually annotated pages, I employed transfer learning by fine-tuning a pre-trained `FasterRCNN_ResNet50`. This is a well-established and powerful architecture for object detection tasks. It consists of the following parts:

- **Faster R-CNN:** This is a two-stage object detection framework. The first stage, a Region Proposal Network (RPN), proposes candidate object bounding boxes. The second stage uses RoI (Region of Interest) Pooling to extract features from these proposals and then classifies them and refines their bounding box coordinates.
- **ResNet50 Backbone:** ResNet (Residual Network) is a convolutional neural network architecture with 50 layers.
- **Feature Pyramid Network (FPN):** FPN is a component that enhances the backbone by creating a feature pyramid with high-level semantic features at all scales. This is particularly beneficial for detecting objects of various sizes, as it allows the model to leverage strong semantic features for small objects and high-resolution features for large objects.

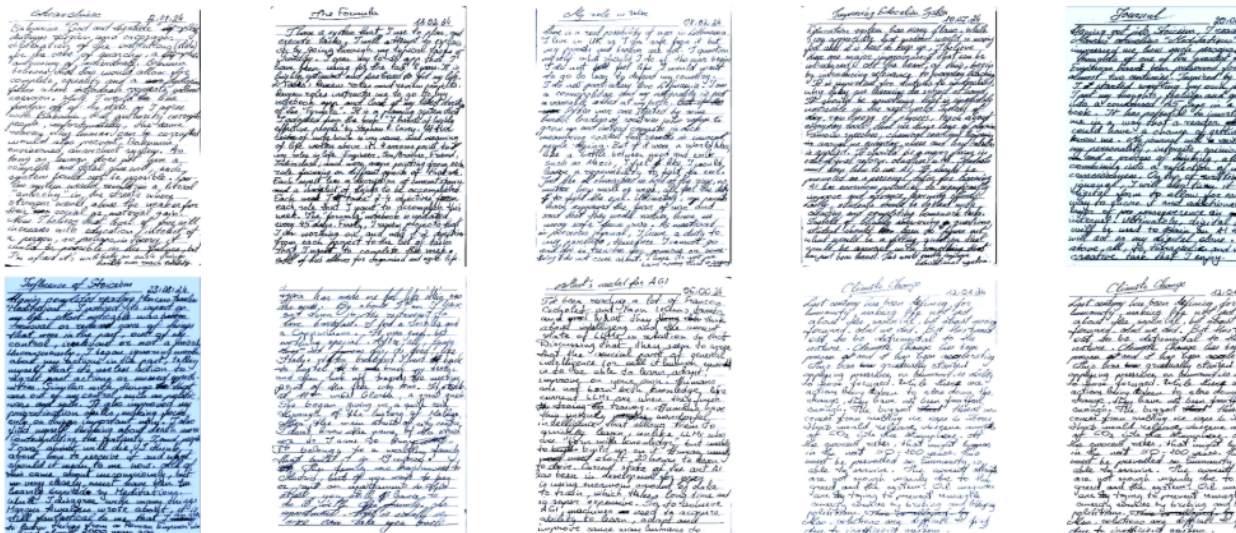
This model has been trained on large-scale datasets and already incorporates robust feature detection capabilities. Its capability to handle variable text sizes aligns perfectly with the challenges inherent in handwritten document segmentation.

Data for Auto-Annotator Fine-tuning

To prepare data for fine-tuning the Faster R-CNN model, images were converted to RGB and the bounding box coordinates and text labels were extracted.

Real-world text in journal scans can have variations in lighting, contrast, and minor distortions. The following augmentations were applied to simulate these real-world conditions:

- ColorJitter - brightness and contrast adjustments.
- GaussianBlur - applies Gaussian blur.
- Normalize - normalizes the pixel values using standard ImageNet mean and standard deviation for more stable model training.



This makes the model more robust and helps it generalize better to unseen images, preventing overfitting to the exact appearance of the training samples. This process is repeated once, effectively tripling the dataset size (from 60 to 180 samples).

The dataset is then split into training and validation sets, with 30% attributed to validation and 70% to training.

Finally, dataloader instances were created with a batch size of 8 and optimised num_workers, persistent_workers, pin_memory, and prefetch_factor parameters to improve PyTorch model training.

Fine-tuning process

The Faster R-CNN model is initialized with pre-trained weights. By using pre-trained weights, the model already has a strong foundation, making it much faster and more effective to train on the new, smaller dataset of journal scans. This also helps prevent overfitting, especially when the target dataset is limited. Fine-tuning allows my system to adapt those weights specifically to the textures and patterns found in the handwritten text.

Training Details

Loss Function

The classification head is replaced with a new FastRCNNPredictor head, adapting it for the specific text detection task. For Faster R-CNN, the overall loss is comprised of the sum of these individual loss components:

- Region Proposal Network (RPN) Losses - Binary cross-entropy loss (or similar) that tells the RPN whether a proposed anchor box contains an object (foreground) or is just background.
- Faster R-CNN (ROI Head) Losses - Binary classification loss (e.g., cross-entropy loss) that classifies the objects within the Region of Interest (RoI) proposals (background and text).

Optimiser

AdamW (Adam with Weight Decay), a popular optimization algorithm was used with a learning rate of 0.0001 and weight decay of $1e-4$.

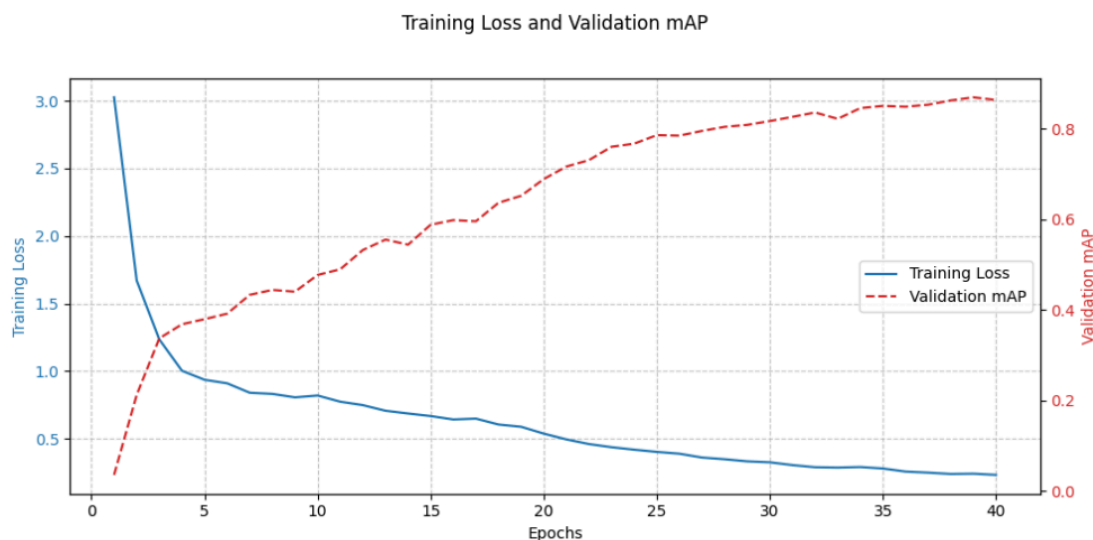
Training

The model is trained for 40 epochs during which the model's weights are adjusted based on the training data, and its performance is evaluated on a validation set using Mean Average Precision (mAP).

The model was trained using GPU P100 available on Kaggle notebooks to speed up the Neural Net training process.

Metrics

mAP (Mean Average Precision) is the most common evaluation metric for object detection tasks. It considers both localization (how well the bounding box is placed) and classification (how accurately the object is identified).



Mixed-Precision Training

Mixed-Precision Training allows the model to perform some operations (like certain matrix multiplications) using lower-precision floating-point numbers (e.g., FP16 instead of FP32). It speeds up training on compatible GPUs and reduces memory consumption. The GradScaler dynamically scales the loss to prevent numerical underflow issues when working with lower precision, ensuring training stability.

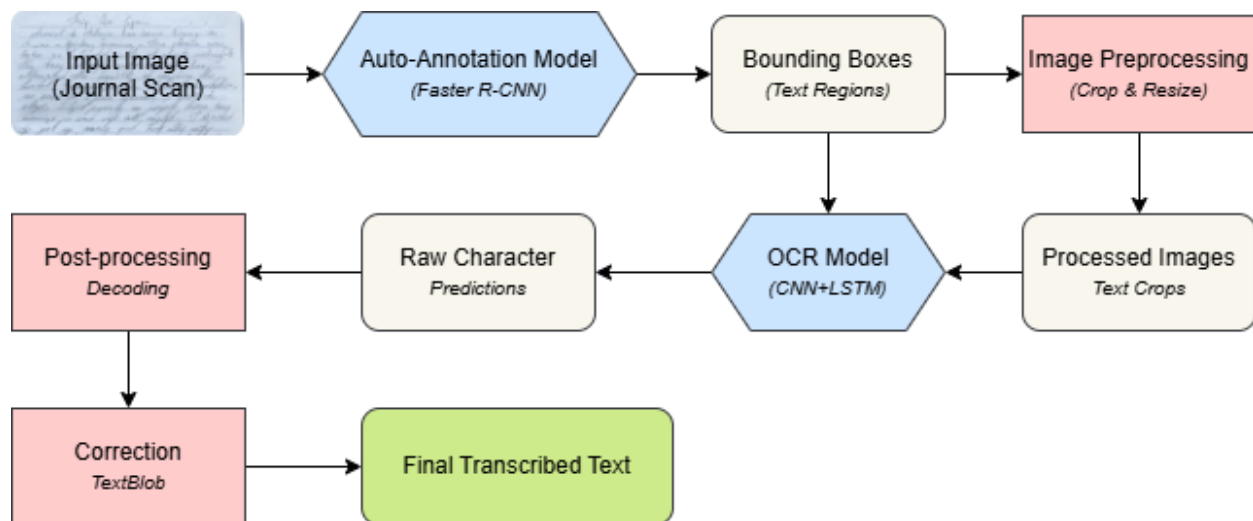
Challenges & Solutions

One of the biggest hurdles in training the auto-annotator model was data scarcity since I pledged to only use my data in this project. Therefore, I chose to fine-tune a pre-trained model. This made the model easier and faster to build and allowed me to practice transfer-learning.

Additionally, a small fine-tuning dataset shows signs of overfitting. To remedy this, more attention was focused on getting the data augmentation and splitting correctly to make the model generalise better.

Inference Pipeline & Deployment: Bringing it to Life

Pipeline Architecture



Both trained models are put into an inference pipeline that processes input images and uses the models to transcribe handwriting from them. The pipeline has been built with a sequential, modular approach. Here is the pipeline execution:

1. Initialise pre-trained Faster R-CNN model and load the fine-tuned state dictionary
2. Define OCR model architecture and load the trained model state dictionary
3. Inference:

- a. Input handwriting image.
- b. Use the annotation model to find and extract bounding boxes in the input image
- c. Crop the image based on the box coordinates. Resize the crop to a fixed height while maintaining the aspect ratio. Convert it to grayscale. Normalize pixel values. Expand its dimensions for model input.
- d. Pad preprocessed image tensors to a uniform maximum width and stack them into a single batch tensor.
- e. Get predictions from the OCR model and decode into readable strings
- f. Join strings into a text.
- g. Replace common errors such as double or triple spaces with single-space.
- h. Correct spelling in the text using TextBlob library.
- i. Output the result.

The use of spelling correction is a crucial step to improve the readability and accuracy of the final transcribed text, addressing common OCR errors that might result in misspelt words. TextBlob was chosen for spelling correction due to its lightweight integration with Python and ability to handle common OCR errors, such as character substitutions.

FastAPI implementation

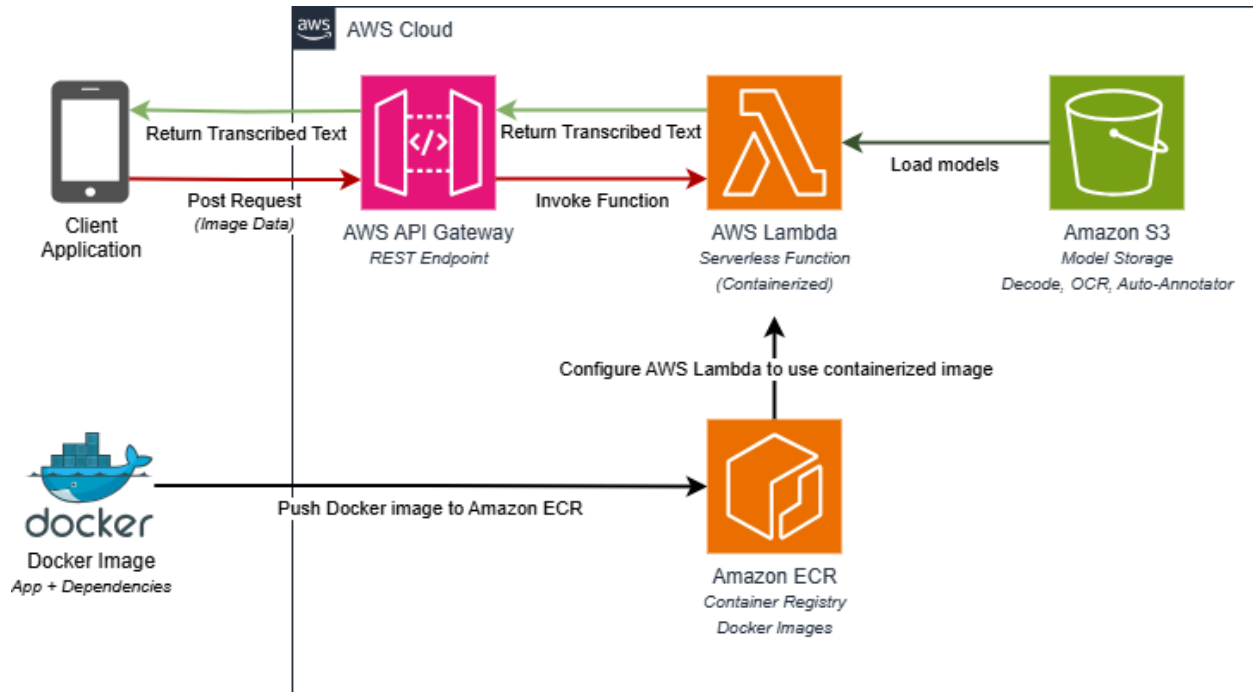
The inference logic was initially encapsulated within a FastAPI application. This modern Python web framework was chosen for its high performance, automatic interactive API documentation (Swagger UI/ReDoc), and ease of development, which significantly streamlined the API creation and testing process locally. This FastAPI application was then adapted into a `lambda_handler` function, specifically designed to run within the AWS Lambda environment in preparation for deployment. This approach allowed me to leverage FastAPI's development benefits before optimizing the code for serverless deployment.

AWS Deployment

The pipeline was deployed on Amazon Web Services (AWS). I used this cloud provider because I already had some experience using it and it provides all the necessary services:

- AWS Lambda: A serverless computing service that is used to run code in response to API requests without provisioning or managing servers. The inference code was deployed as a containerized image using Docker.
- Amazon Elastic Container Registry (ECR): A fully managed container registry service used to store, manage, and deploy Docker container images securely.
- Amazon S3: A scalable storage service where the encoder, annotation, and OCR models are stored. Lambda accesses these models for processing handwriting transcription.
- Amazon API Gateway: A fully managed service that creates RESTful APIs. It facilitates communication between external applications and the Lambda function, allowing users to send POST requests for transcription.

AWS Lambda was chosen over other options such as EC2 or SageMaker services. EC2 resulted in higher costs and management overhead. SageMaker similarly culminated in higher cost and complexity of the setup. Lambda's cost-efficiency for sporadic use and my previous experience using the service made it the best option for this scenario.



The deployment included the following steps:

1. Created a Docker image containing the application code and dependencies.
2. Pushed the image to Amazon Elastic Container Registry (ECR).
3. Configured Lambda to run the containerized application.
4. Stored the encoder, annotation, and OCR models in an S3 bucket.
5. Configured API Gateway to expose a REST endpoint.
6. Routed POST requests to the Lambda function.

Deploying PyTorch and its dependencies exceeded AWS Lambda's 250 MB unzipped size limit, causing deployment failures. This delayed the deployment phase and risked abandoning serverless architecture. To fix this I Containerized the application using Docker. Furthermore, I optimized the container by removing unnecessary dependencies. Then deployed the pipeline as a containerized Lambda function, achieving scalability and cost-efficiency.

User Interface: Making it Usable

iOS Shortcut Application

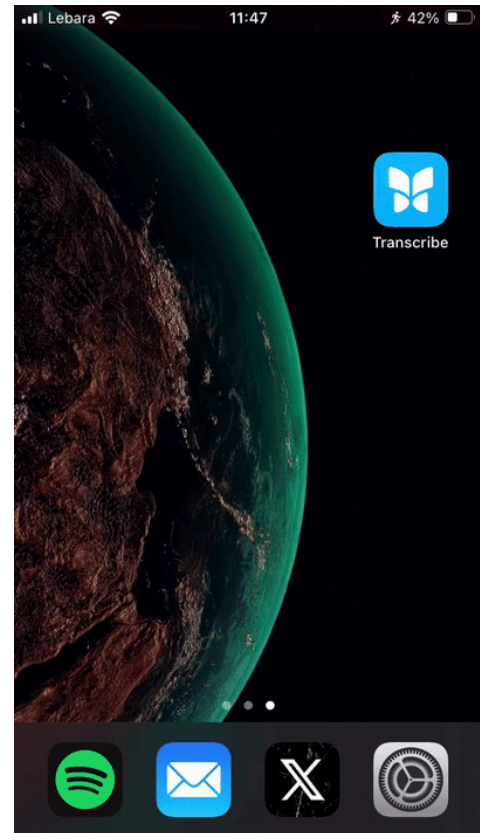
To create a user-friendly interface that simplifies interactions with the handwriting transcription API, I implemented an iOS Shortcut app. By leveraging native iOS capabilities, the app streamlines the process from image capture to displaying the transcribed text. The design focus is on ease of use while ensuring that the necessary processing steps are handled efficiently on the device before sending data to the cloud.

Step-by-Step process performed by the application:

1. Capturing the Image with the back camera of the device
2. Resizing the Photo
3. Encoding the Image to Base64
4. Sending the POST Request
5. Unwrapping and Decoding the API Output
6. Displaying the Result

Benefits:

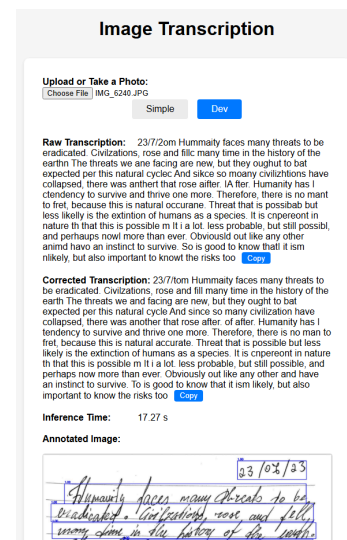
- Ease-of-Use: The use of an iOS Shortcut provides easy interaction with complex backend processing without any visible latency or cumbersome UI elements.
- Seamless Integration: The app leverages Apple's native tools, ensuring compatibility and a smooth user experience.



Web-based API Testing Tool

To further facilitate testing, demonstration, and debugging of the deployed API, I made a simple web interface. This tool allows users to:

- Upload an input image: Easily select and submit handwritten journal pages for transcription.
- Perform POST request: Initiate the transcription process by sending the image to the deployed AWS API Gateway endpoint.
- Display comprehensive output: The results include the raw transcription from the OCR model, the spelling-corrected transcription, the inference time for the entire process, and



the input image with the detected bounding boxes overlaid. This visual feedback is invaluable for assessing the auto-annotation model's performance and identifying potential issues with text segmentation.

The tool was hosted on a local machine to test the API before AWS deployment. Later the web-based tool was deployed as a static website using AWS S3 and AWS Cloudfront to test the deployed API.

This web tool served as a critical component during development and validation, providing immediate visual and textual feedback on the API's performance, allowing for quick iteration and refinement.

Results & Performance

The OCR model can transcribe most of the handwriting, but its limited generalisation occasionally results in errors:

Character Error Rate: 2.3%

Word Error Rate: 9.33%

Sample 1:
Ground Truth: and respectful.
Prediction: and respectful.



Sample 2:
Ground Truth: one
Prediction: one



Sample 3:
Ground Truth: friends broke
Prediction: friends broke



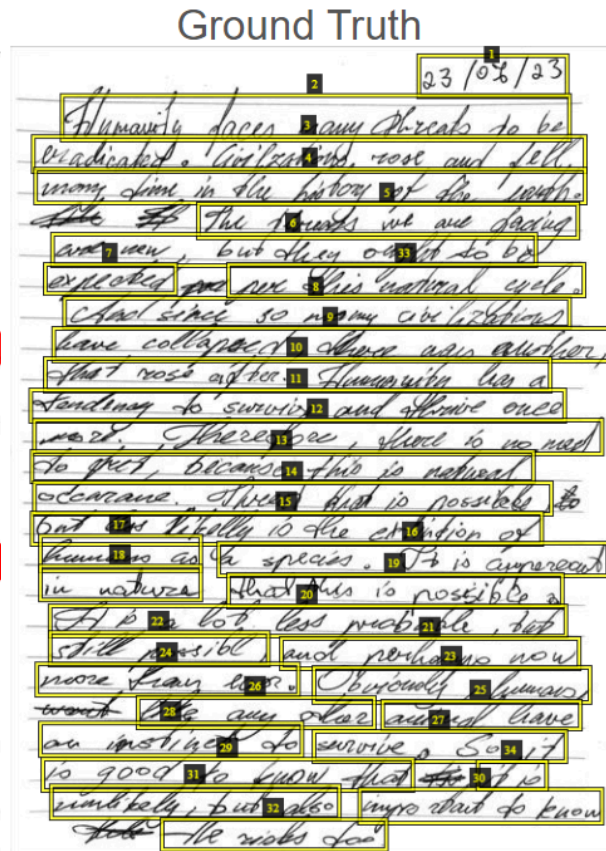
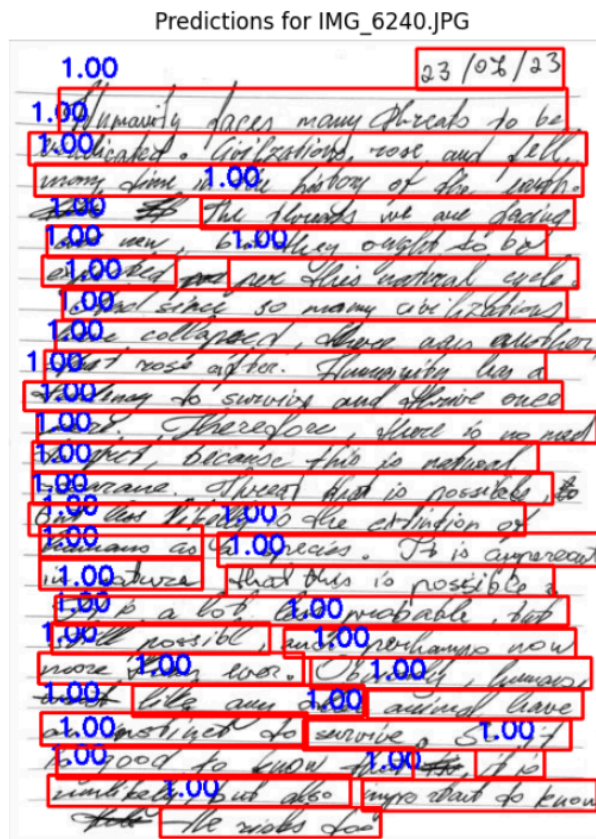
Sample 4:
Ground Truth: comparison is ok to
Prediction: comparison is ok to



Sample 5:
Ground Truth: needed by humans to
Prediction: needed by humans to



The Auto-Annotation model also works very well making predictions with high confidence, but occasional bounding boxes tend to cut off some text in the image, which negatively affects OCR model performance.



The inference speed averages around 18 seconds using the CPU. This is a nearly 17x improvement compared to an average of 5-minute manual transcription.

Challenges, Learnings & Future Scope

Key Challenges

The main challenges in completing this project included the following:

- Lack of advanced knowledge - Having no experience working with PyTorch, using my training data, and training such models has slowed down the progress of the project, making me spend more time learning and practising skills needed to perform the tasks.
- Limited dataset size - The commitment to only using my own data for this project challenged me to find ways to complete tasks with limited-size data.

The overall project spanned approximately 9 months, a duration influenced by several concurrent commitments and unforeseen learning curves. This extended timeline was primarily due to:

- A significant initial learning phase, as I gained proficiency in PyTorch and deep learning model development from scratch.

- The highly laborious and time-consuming manual annotation process, which alone took 3 months.
- Balancing the project alongside my demanding MSc Data Science coursework.
- Managing a part-time job concurrently with the project and studies.

Solutions & Problem-Solving

Learning how to complete tasks of this project involved a lot of research into existing examples of similar tasks and experimenting with my dataset to find what works. Many approaches ended up in failures slowing down the progress, but advancing my knowledge and understanding of what needs to be done. I spent a lot of time trying to train a model that takes a full journal entry page as input and transcribes it without decoupling it into bounding boxes of words and sentences. I experimented with many different architectures, but the performance would not improve, mainly due to the limited size of the dataset. While this did not progress the project, I have learned a lot in the process, and this allowed for better and faster training of the OCR model trained later. The use of LLMs such as Claude Sonnet 3.5 made the learning process much faster and more engaging.

The limited size of the dataset meant that a lot of attention would have to be focused on effective data preparation, especially augmentations, as well as careful selection of model architectures that would work even with limited amounts of data. Having experimented with many different techniques and methods of data augmentation and model architectures, I was able to find combinations that allowed me to complete the project.

The agile and iterative process of experimentation allowed for continuous learning and adaptation, speeding up the progress of the project while managing work and study commitments on the side. Leveraging online resources and LLMs helped accelerate my understanding and problem-solving. Maintaining consistent, focused effort over the extended period, demonstrated perseverance and effective long-term project management skills.

Key Learnings

This project was an intensive learning experience, providing practical insights and strengthening my capabilities across several domains:

- Deep Learning & Frameworks: Deepened proficiency in PyTorch, including the practical application and nuances of Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTMs) for sequence-to-sequence problems like OCR.
- Model Development Strategies: Understood the critical importance of robust data augmentation for improving model generalization with limited datasets, and the practical benefits of transfer learning for accelerating development.
- End-to-End Pipeline & Cloud Deployment: Gained hands-on experience in building a complete AI pipeline, from model training and optimization to AWS deployment (Lambda, ECR, S3, API Gateway), including navigating challenges such as dependency

management and resource limits in serverless environments. Enhanced skills in debugging complex distributed systems.

- Project Management & Problem-Solving: Developed a stronger appreciation for the agile, iterative process in technical projects, learning to experiment, analyze failures, and adapt strategies effectively.
- Data & Annotation: Reinforced the paramount importance of data quality and accurate annotation as the foundation for successful machine learning models.
- Modern Development Tools: Learned to effectively leverage Large Language Models (LLMs) as powerful aids for accelerating research, debugging, and skill acquisition.

Future Improvements

- Collect more data / use data augmentation more extensively
- Replace Faster R-CNN with better performing model
- Try more advanced model architectures (e.g., Transformers).
- Optimise inference time
- Switch inference to GPU-backed service for faster response
- Language model integration for error correction.
- Blog auto-upload pipeline

Conclusion

This project successfully delivered a handwriting transcription pipeline, transforming personal handwritten journal entries into digital text. I designed and built an Optical Character Recognition (OCR) model from scratch using a CNN-LSTM architecture in PyTorch, demonstrating proficiency in deep learning model development, data preparation, and extensive augmentation techniques. Furthermore, I built an auto-annotation model by fine-tuning a pre-trained Faster R-CNN for efficient text region detection. The entire pipeline was then deployed on AWS Lambda using Docker containers, ECR, S3, and API Gateway, showcasing practical cloud deployment skills. Finally, I created a user-friendly iOS Shortcut application to streamline the image capture and transcription process.

This endeavour significantly enhanced my practical skills in PyTorch, Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), and transfer learning. Beyond the technical growth, this project directly benefits me by digitizing a year of personal journal entries, making them easily shareable on a blog, a process that would have been incredibly tedious and time-consuming if done manually. The developed system offers a remarkable 17x improvement in transcription speed over manual methods, freeing up valuable time and effort. This project not only validated my academic learning in data science but also provided invaluable real-world experience in building, deploying, and optimizing an end-to-end AI solution.

I'm keen to apply these skills to similar challenging problems. If you're looking for someone with hands-on experience in deep learning, computer vision, and cloud deployment to tackle your data science initiatives, let's connect!